NAMMA SPACE · TECHFEST 2026-27 · IIT BOMBAY · ROUND 1: 3D RECONSTRUCTION & ROAMING

Wayfarer: from a smartphone video to a walkable indoor digital twin

An end-to-end, one-command pipeline that turns an ordinary phone walkthrough into a photorealistic, metric, gravity-aligned 3D twin you can explore in first person in any browser, with nothing to install.

TEAM NAME	TEAM ID	INSTITUTE
----------	----------	----------

TEAM MEMBERS

CODE REPOSITORY
github.com/MNNITAK/Wayfarer

LIVE WEB VIEWER ----------

DEMO VIDEO ----------

1 cmd
video → web twin, every stage resumable

97 %
frames registered on our home capture (284 / 292, one model)

~1 %
metric scale error with a printed ArUco marker

~11×
smaller splat after SOG compression (full SH colour kept)

0
installs or manual 3D modelling for the visitor

1 Overview & system architecture

Wayfarer has three parts, one per actor. **(1) Capture**: a phone-based **Capture Coach** turns our SOP into live guidance and checks each take before anything heavy runs. **(2) Process**: a local job server runs the reconstruction pipeline on one GPU PC (or a free Kaggle GPU): blur-aware frame selection, COLMAP Structure-from-Motion, 3D Gaussian Splatting, then our own **navigation-grid builder**, which levels the model, gives it real-world scale and extracts a walkable occupancy map. **(3) Experience**: a static web viewer streams the compressed splat and lets anyone roam in first person with true wall collision. The same metric grid drives POIs, search and A* routing for Round 2, and anchors AR navigation in the finale.

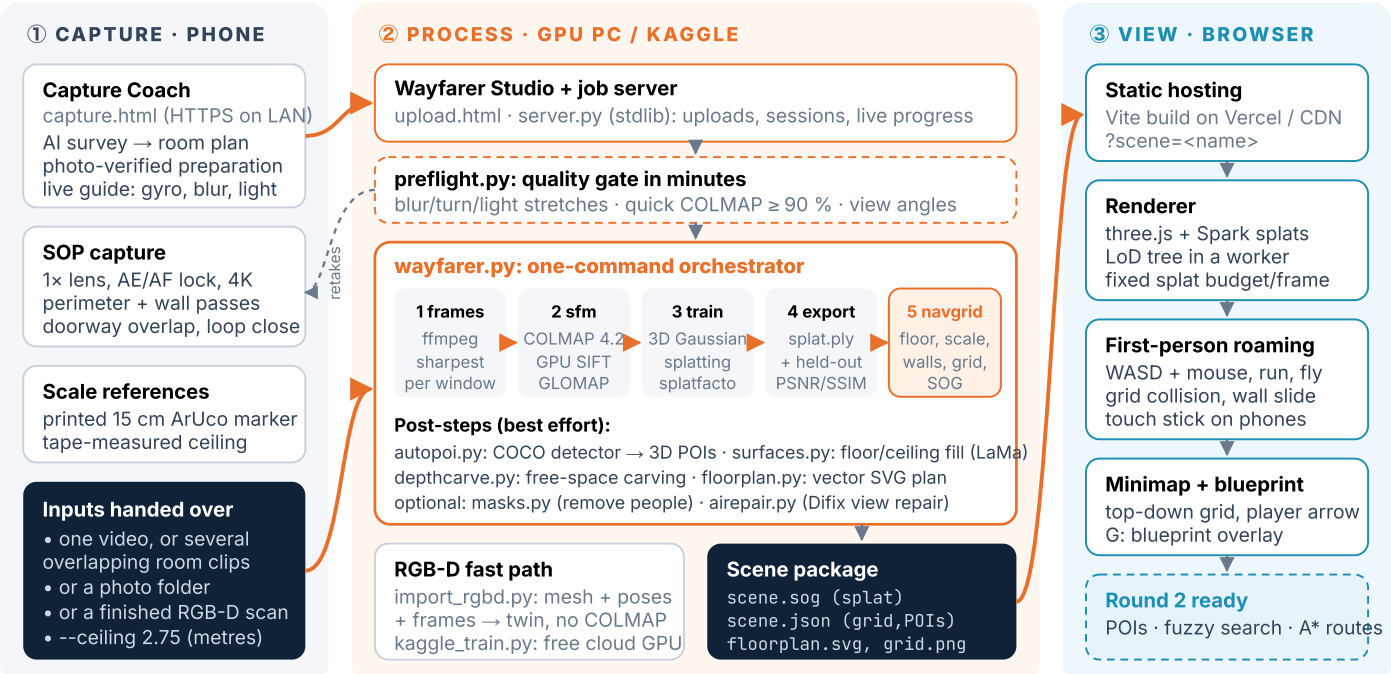


Figure 1. System architecture. Orange marks our own components; the rest wraps established open-source tools (COLMAP, nerfstudio/gsplat, Spark). Each stage writes its output to disk and is skipped when the output exists (`--force <step>` re-runs from a step), so a crashed 30-minute job resumes where it stopped.

2 Capture SOP: every rule tied to the failure it prevents

Reconstruction quality is decided at capture time. Our SOP (`docs/SOP.md`) is written for a non-expert with a phone and states the reason next to each rule. Its numbers come from one source, `pipeline/capture_requirements.md` , which the Capture Coach and the automatic checks both read.

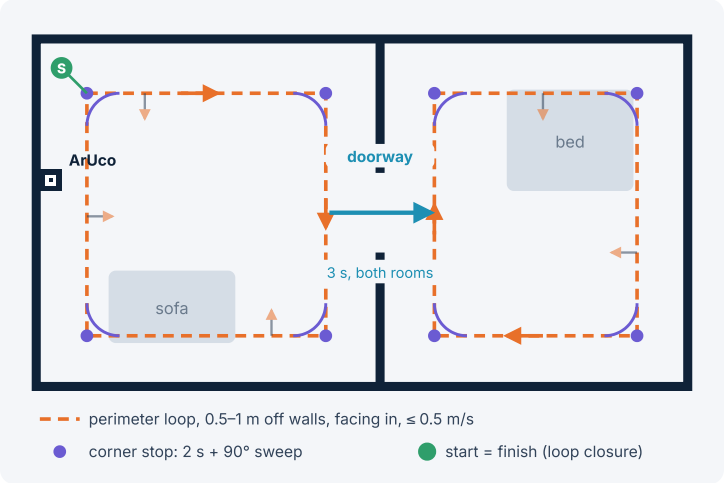


Figure 2. Capture path (top view). Each room gets a perimeter loop, corner sweeps, one ceiling sweep and along-the-wall passes. Rooms are tied together by walking slowly through doorways.

SOP rule	Failure it prevents
1× main lens, landscape, AE/AF lock, HDR/night off	Lens distortion; brightness pumping that breaks photometric consistency
Walk ≤ 0.5 m/s, turn ≤ 30°/s, chest height ~1.4 m	Motion blur, too little parallax for SfM
Tilt 10–15° down; one ceiling sweep per room	No floor/ceiling evidence, which leveling and scale need
Along-the-wall pass, 30–45° to the wall	Walls seen only head-on turn fibrous when you walk along them
Never film a blank wall from < 1 m	No features → lost tracking
Fans off, mirrors/screens covered, doors open	Floaters; "phantom rooms" behind reflections
Doorway overlap 3 s; end where you started	Rooms that fail to stitch; drift
15 cm ArUco marker from 3+ angles; tape the ceiling	Scale ambiguity of monocular SfM

Two modes: one long take (homes, ≤ ~8 min) or room-by-room clips that start and end in a doorway. Frames from all clips are matched together, so the clips register into one model **without manual stitching**.

2.1 Capture Coach: the SOP as an assistant, and a gate before the long build

A bad take used to show up only after a 30-minute build. The Coach (`capture.html` , opened on the phone) keeps the user in a loop until the capture meets the pipeline's needs.

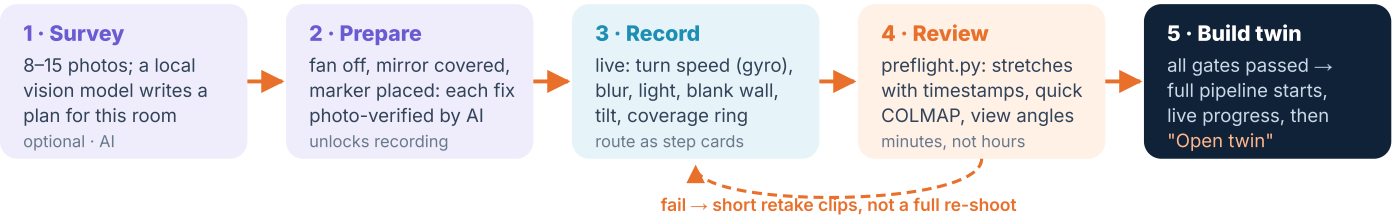


Figure 3. Capture Coach loop. The AI steps run on a free local vision model (Ollama, `qwen2.5vl:7b`), so photos never leave the PC; the Claude API is optional. Without either, steps 3–5 still run.

2.2 Quality gates (automatic, with the numbers the checks use)

Check	Threshold	Runs in	If it fails
Motion blur	sharpness < 0.55× best frame within ±2 s	guide · preflight · picker	walk slower, more light
Turn / exposure	> 30°/s · grey jump > 18 in 1/8 s	guide (gyro) · preflight	retake that stretch
Dark / blank wall	grey < 35 · gradient < 1.5 levels/px	guide · preflight	lights on, keep an edge in view
Camera poses	≥ 90 % in one model · build stops < 50 %	preflight · `step_sfm`	more doorway overlap
View angles	median ray spread ≥ 15° per plan cell	preflight	along-the-wall retake
Scale cross-check	marker / ceiling / camera within 12 %	`navgrid.py`	re-film marker, re-measure

3 Reconstruction pipeline: video to photoreal 3D

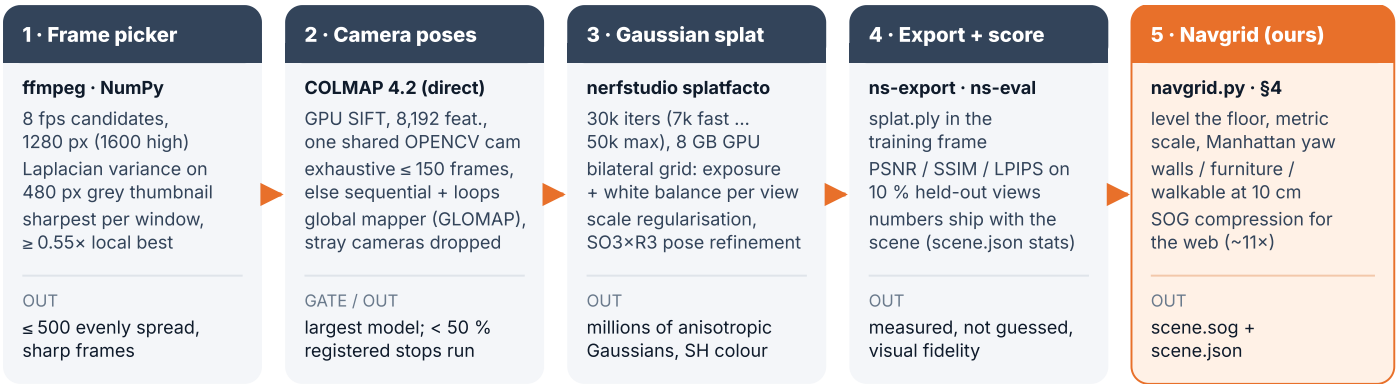


Figure 4. Pipeline stages (`python pipeline/wayfarer.py walk.mp4 --name home --ceiling 2.75`). Capture statistics and timings are written to `stats.json` and travel with the twin.

3.1 Blur-aware frame selection

Video is sampled at 8 fps and thinned to at most 500 frames by keeping the sharpest frame in each window (variance of the Laplacian). A frame is rejected only if it falls below **0.55× the sharpest frame within ±2 s**, not below a global median: a plain wall is naturally low-contrast yet sharp, while motion blur is low *relative to its neighbours*. Photo folders keep every photo.

3.2 Structure-from-Motion

We drive COLMAP directly, not through nerfstudio's wrapper, which turns loop detection off and only reads `sparse/0`; a multi-room walk could then drift or silently lose rooms. Short captures use **exhaustive matching**, so every revisit closes a loop. Longer ones use sequential matching (overlap 20, quadratic) plus vocabulary-tree loop closure, which is O(n). Mapping uses COLMAP 4.2's **global mapper (GLOMAP)**, which solves all cameras at once. On our home capture it registered **284 / 292 frames in one model at 0.67 px reprojection error in 6.5 min**, against 282 frames split into two models in 17 min with incremental mapping. Cameras the mapper placed implausibly far away (> 10× the median distance) are deleted; otherwise they squash the real poses and fill the room with fog. Matches are cached by a hash of the frame list, so an interrupted run resumes.

3.3 3D Gaussian Splatting

Splats were chosen over meshes and NeRFs because they render **photoreal, view-dependent appearance in real time in WebGL**, and they hold up on thin and glossy structures (chair legs, plants, tiles) where photogrammetry meshes melt. We train `splatfacto` (gsplat CUDA kernels), initialised from the SfM points, with settings that fit a multi-room scene in **8 GB**: images cached on the CPU as uint8, a per-image **bilateral grid** that absorbs exposure and white-balance shifts between rooms, scale regularisation against needle splats, and SO3×R3 camera refinement. Interrupted runs resume from the last checkpoint. An MCMC trainer (`train_mcmc.py`) with a hard Gaussian cap is kept as a memory-safe fallback for very large venues.

3.4 Making the twin complete and clean

- **Floor & ceiling fill** (`surfaces.py`): chest-height footage barely sees these planes. We project real pixels from the frames onto the known planes at 2 cm per texel, occlusion-checked against rendered depth, and use LaMa inpainting only for what no frame saw.
- **People removal** (`--mask-people`): masked pixels are ignored by both COLMAP and training, so other frames supply the real background with no ghosts.
- **AI view repair** (`--ai-repair`, Difix3D+ approach): novel views are rendered, repaired by a one-step diffusion model, and fine-tuned back into the 3D.
- **RGB-D fast path** (`import_rgbd.py`): a finished scan (mesh + metric poses + frames) becomes a twin in about a minute with no GPU; `kaggle_train.py` trains the photoreal splat on a free Kaggle GPU.

4 Navigation-grid builder (our core contribution)

A trained splat is just a cloud of coloured ellipsoids in an arbitrary SfM frame: it has no floor, no scale and no notion of what is solid. `navgrid.py` turns it into a **level, metric, walkable map**.

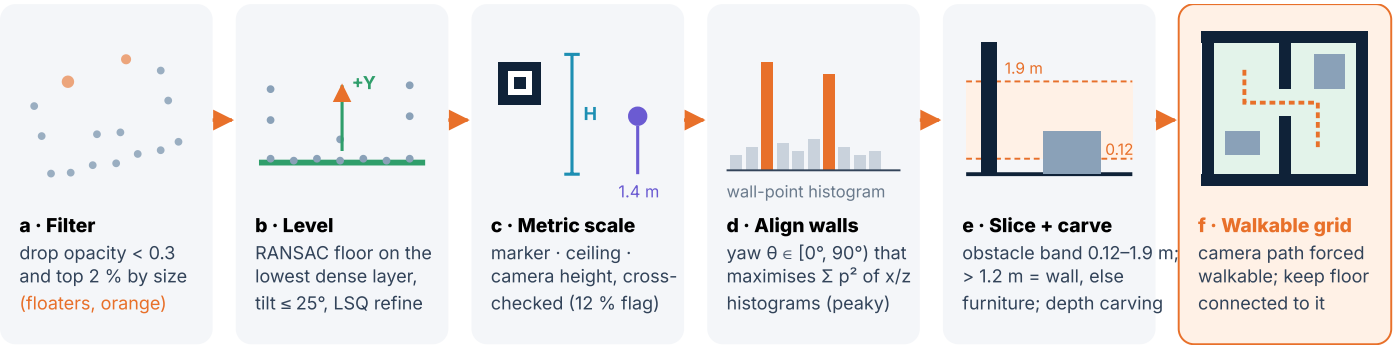


Figure 5. From splat cloud to navigation grid. Output cells: `.` walkable, `#` wall, `o` furniture, blank = unknown, 10 cm resolution.

- **Floor**: RANSAC over the lowest dense layer (not simply the largest plane, which is often a wall or table top), with normals within 25° of the up hint, then refined to the smallest-eigenvalue axis of the inliers. The scene is rotated so the floor normal is +Y and the floor sits at y = 0. If the up hint is 90° off (ceiling-heavy footage), the camera path still recovers the floor.
- **Obstacles**: splats between 0.12 m and 1.9 m, weighted by opacity, are binned into 10 cm cells with a density-adaptive threshold. Blobs under 0.03 m² are dropped as floaters.
- **Free-space carving** (`depthcarve.py`): expected depth is rendered from the training cameras and each ray is marched through 10 cm voxels in the obstacle band. Voxels that rays passed *through* far more often than they stopped in are floaters and get cleared; ray endpoints add surfaces. Working per voxel matters: a ray passing above a bed says nothing about the bed.
- **Walkability**: floor evidence is closed morphologically. Cells the **camera physically walked through** are always walkable (so doorways stay open), and only floor connected to that path is kept, which removes "rooms" seen through windows and mirrors.

Three independent scale estimates

$$s_m = 0.15\ m \,/\, \| corner_i - corner_j \|$$

ArUco corners triangulated (DLT) from the COLMAP poses; needs 3+ views with sides agreeing within 5 %

$$s_c = H_{tape} \,/\, d(floor, ceiling)$$

ceiling = second RANSAC above every camera

$$s_h = 1.4\ m \,/\, median\ camera\ height$$

Priority: marker > ceiling > camera; any estimate > 12 % off is flagged

Web export: PlayCanvas *splat-transform* removes NaN Gaussians and compresses to **SOG** (k-means-quantised attributes in lossless WebP, about 10 bytes per splat including all SH bands): a 90 MB PLY became 8 MB. The alignment is stored as a TRS transform in `scene.json`, so the splat is never resampled, and the grid is plain text that can be fixed in any editor.

5 Zero-install web viewer

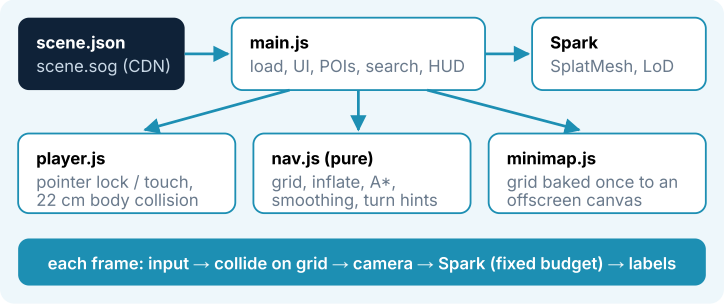


Figure 6. Viewer modules. Static Vite build: an HTML page plus two data files, so any CDN can serve it.

- **Smooth on ordinary GPUs**: Spark builds a level-of-detail tree in a worker, so each frame draws a **fixed splat budget** whatever the scene size. Splats are clipped at √5σ instead of √8σ to save fill rate, the full-detail cone is widened to 130° so screen edges stay sharp, pixel ratio is capped (lower on phones), and labels are DOM overlays with no extra draw calls.
- **Game-like movement**: WASD with exponential acceleration and damping, Shift to run, slight head-bob, `F` fly mode for a dollhouse view. Movement is resolved per axis, so you **slide along walls** instead of sticking.
- **Works without a splat**: the grid is extruded into a blueprint, so a scene can be checked before training ends.

6 Validation & results

Test	Result
Own home capture (one take, SOP-compliant), COLMAP 4.2 global mapper	284 / 292 frames registered (97 %) in **one** model, 0.67 px mean reprojection error, 6.5 min. Incremental mapping: 282 frames split in two models, 17 min.
Synthetic splat flat (6×4 m, 2.6 m ceiling, door, table) moved by 15° yaw, 2° tilt, 3× scale	Scale recovered as 3.001 (**0.04 %** error); ceiling and camera estimates agree (3.0012 vs 3.0015); walls axis-aligned; door walkable
ArUco marker rendered from 5 known poses	1.0028 m/unit vs a true 1.0000 (**0.3 %**); fewer than 3 views correctly refused
Wrong up hint (90° off, as ceiling-heavy footage gives)	Floor still recovered from the camera path; scale within 3 % (regression test)
Public RGB-D benchmark (Redwood indoor apartment, `import_rgbd.py`)	Walkable twin with 4.7 M splats (28.6 MB SOG), 41.6 m² walkable, **18 POIs labelled automatically** (4 rooms + bed, sofa, fridge, oven, TV…)
Trainer benchmark, 77-photo flat, 30k steps	splatfacto 27.72 dB vs gsplat-MCMC 23.09 dB PSNR in the same time: splatfacto is the default
Automated tests: `npm test` + 4 Python self-checks	Pathfinding, collision, turn hints, live capture metrics; navgrid, marker scale, preflight, coach (stubbed AI)
Home twin quality (held-out views) & viewer FPS	PSNR __ dB · SSIM __ · LPIPS __ · __ M splats · __ MB · __ FPS on GPU / laptop

7 Built for Rounds 2 & 3

- **POIs**: auto-labelled from a COCO detector (Faster R-CNN), placed in 3D from depth, clustered, kept only when seen from 3+ views; rooms are inferred from contents (a bed makes a bedroom). Press `T` to tag at the crosshair; labels reveal on click or gaze.
- **Search**: ranked fuzzy match (exact > prefix > word-prefix > substring > subsequence) with distances.
- **Routing**: 8-connected A* (octile heuristic, no corner cutting) on a grid inflated by a body margin, falling back to the raw grid for tight doors; line-of-sight smoothing; animated floor chevrons; re-plans as you walk.
- **AR (finale)**: the twin is metric and gravity-aligned, so WebXR needs only one rigid transform from phone tracking to twin (QR anchor at the entrance); the A* route becomes arrows on the real floor.

8 Reproduce it

```
# one-time setup: docs/SETUP.md (COLMAP, ffmpeg,
nerfstudio)
cd web && npm install && npm run dev
#  → localhost:5173 (procedural demo home)

# rebuild a twin from the sample capture
python pipeline/wayfarer.py test/sample/walk.mp4 \
       --name sample --ceiling 2.75
#  → localhost:5173/?scene=sample

# or: double-click start.bat → Wayfarer Studio,
#     drop the video, press "Build twin"

# tests
cd web && npm test
python pipeline/test_navgrid.py
```

Sample data for judges: `test/` in the repository add path / Drive link. Hardware used: RTX 4070 Laptop (8 GB) or a free Kaggle GPU.

9 Technology stack & open-source credits

Layer	Tools (licence)	Our original work
Capture	Ollama + Qwen2.5-VL (Apache-2.0), optional Claude API; browser DeviceMotion	Capture Coach, live checks, preflight gate, retake planner
Reconstruction	FFmpeg (LGPL), COLMAP / GLOMAP (BSD), nerfstudio, gsplat (Apache-2.0), OpenCV ArUco (Apache-2.0), LaMa (Apache-2.0), torchvision (BSD), Difix3D (NVIDIA, research use)	Orchestrator, frame picker, stray-camera filter, navgrid builder, marker scale, depth carving, surface fill, auto-POIs
Web	three.js, Spark, PlayCanvas splat-transform, Vite (all MIT)	Viewer, player & collision, minimap, POIs, search, A* navigation